

IT5030 – Software Engineering Practices

Sprint 1 Report

20th February – 6th March 2026

Car Sales and Dealership Management System

Group 10

P.M.A.D.N. Kulathunga	MS25937152	Product Owner
B.V.S.M. Athurupana	MS25944228	Scrum Master
R.R.M.G.H. Rathnayake	MS25943610	Frontend Developer
H.M.S.N. Rajapaksha	MS25937848	Backend Developer
G.G.P.S. Gunathilaka	MS24900904	Full-Stack / QA

Contents

1	Sprint Overview	2
2	Sprint Backlog	2
3	Sprint Activities and Implementation	2
3.1	Development Environment Setup	2
3.2	User Authentication System (US01)	3
3.3	Vehicle Inventory – Add Vehicle (US02)	3
3.4	Vehicle Inventory – View and Search (US03)	3
3.5	Vehicle Inventory – Edit Vehicle (US04)	3
3.6	Database Schema Implementation	3
3.7	Dashboard Layout	4
4	Testing Activities and Results	4
4.1	Test Summary	4
4.2	Key Test Cases	4
5	Scrum Artifacts	5
5.1	Task Board Summary	5
5.2	Product Backlog Updates	5
6	Individual Contributions	5
7	Sprint Review	5
7.1	Completed Features	5
7.2	Incomplete / Carried Forward	6
7.3	Demo Feedback	6
8	Sprint Retrospective	7
8.1	What Went Well	7
8.2	What Could Be Improved	7
8.3	Action Items for Sprint 2	7
9	Sprint 2 Preview	8
9.1	Sprint 2 Goal	8
9.2	Planned User Stories	8

Sprint Overview

Item	Details
Sprint Number	Sprint 1 – Core Foundation
Duration	20 th February – 6 th March 2026 (2 weeks)
Sprint Goal	Implement user authentication, vehicle inventory CRUD, database schema, basic dashboard layout, and API structure
Total Story Points Planned	18 points
Total Story Points Completed	15 points
Sprint Velocity	15 points / sprint

Table 1: Sprint 1 Summary

Sprint Backlog

The following user stories were selected for Sprint 1 during the Sprint Planning ceremony:

ID	User Story	Priority	Points	Status
US01	As an admin, I want to register and log in securely so that I can access the system based on my role	Must	5	Done
US02	As a sales agent, I want to add new vehicles to the inventory with all specifications so that listings are up to date	Must	5	Done
US03	As a sales agent, I want to view, search, and filter vehicle listings so that I can quickly find cars matching customer needs	Must	5	Done
US04	As a sales agent, I want to edit and update vehicle details and status so that inventory reflects current availability	Must	3	In Progress

Table 2: Sprint 1 Backlog Items and Status

Sprint Activities and Implementation

Development Environment Setup

The team configured the development environment during the first two days of the sprint. This included initialising the monorepo structure with separate `client/` and `server/` directories, configuring TypeScript for both projects, and setting up the PostgreSQL database with Prisma ORM. Atlassian Jira was adopted to monitor tasks.

- Initialised React + Vite + TypeScript frontend with Tailwind CSS and shadcn/ui
- Set up Node.js + Express + TypeScript backend with ts-node-dev for hot reloading

- Configured Prisma schema and ran initial migration to create all database tables
- Created seed script with sample users, vehicles, and customers
- Configured ESLint + Prettier for consistent code formatting across the team
- Set up Git repository with feature branching strategy on GitHub and unified Git branches
- Scaffolded local containerization with Docker (PostgreSQL and pgAdmin)

User Authentication System (US01)

The system authentication has been fully realised. We engineered a secure, robust login paradigm driven by JSON Web Tokens (JWT) distributed via HTTP-only cookies. Passwords undergo rigorous, cryptographically secure hashing utilizing `bcrypt` before insertion into the PostgreSQL relational tier. Role-based access control (RBAC) definitions map user profiles directly to four operational groups (Admin, Manager, Sales Agent, Inventory Staff) enabling conditional route guards on the React Frontend (AuthContext Provider).

Vehicle Inventory – Add Vehicle (US02)

Integration between the Client application and Backend database mapping inventory entries was successfully achieved. Administrators and Sales Agents have an interface to populate the `Vehicles` data entity via sophisticated form interfaces. Using `Zod` schemas on the frontend minimizes latency associated with request validation before hitting standard Express `POST` endpoints, enforcing strict integrity protocols over inputs (Make, Model, Year, VIN, Price).

Vehicle Inventory – View and Search (US03)

An indexing overview grid was programmed onto the React components integrating `shadcn/ui` table designs. The view enables dynamic toggling between List and Grid interfaces. Extensive API route queries power the seamless pagination, textual cross-referencing capabilities, and filtering operations by fuel type, price, and operational status yielding high-performance vehicle tracking matching the specifications precisely.

Vehicle Inventory – Edit Vehicle (US04)

The backend REST endpoint (`PATCH`) for modifying vehicle records was completed. However, the frontend form integration encountered delays due to complex state management issues regarding image unlinking and concurrency handling. The basic text fields can be updated, but file handling edge cases caused this story to be somewhat delayed and carried over.

Database Schema Implementation

The database schema was implemented using Prisma ORM coupled with the PostgreSQL cloud container. The architecture mandates rigorous relational integrity encapsulating seven foundational entities: Users, Vehicles, Customers, Sales, Appointments, Invoices, and ActivityLog. Automated SQL migrations were standardized for local setups.

Dashboard Layout

A highly interactive foundational user interface layout was successfully accomplished. It features an adaptive and responsive sidebar navigation panel. Dynamic role-based visibility conditions securely display or shroud analytical dashboard components respectively based on the logged-in user profile, establishing a polished administration zone capable of tracking fleet performance statistics uniformly.

Testing Activities and Results

Test Summary

Module	Test Type	Total	Passed	Failed
Authentication	Unit / Integration	10	10	0
Vehicle CRUD	Unit / Integration	12	10	2
API Endpoints	Integration	8	8	0
Frontend Forms	Component	5	3	2

Table 3: Sprint 1 Test Results Summary

Key Test Cases

- **Auth – Login with valid credentials:** Token validated securely and redirects Admin to targeted dashboard. **Result: Passed.**
- **Auth – Login with invalid credentials:** Backend asserts rejection correctly logging a localized 401 Unauthorized status displaying specific GUI error parameters gracefully. **Result: Passed.**
- **Auth – Access protected route without token:** Middleware precisely filters non-authorized ingress requests rerouting seamlessly back to central standard authentication portals. **Result: Passed.**
- **Vehicle – Create with valid data:** New Prisma instantiation securely tracks records syncing immediately inside PostgreSQL displaying green prompt banners. **Result: Passed.**
- **Vehicle – Search by make/model:** Index matches regex arrays resolving non-case sensitive variations. **Result: Passed.**
- **Vehicle – Edit with image files:** Updating state with multiple removed and new images results in desync. **Result: Failed (Carried Over).**

Scrum Artifacts

Task Board Summary

Category	To Do	In Progress	In Review	Done
Backend Tasks	0	0	1	9
Frontend Tasks	0	2	1	7
Testing Tasks	0	1	0	4
Documentation	0	0	0	4
Total	0	3	2	24

Table 4: Sprint 1 Task Board at Sprint End

Product Backlog Updates

During sprint progression, tickets mapping user interaction elements and API endpoints were broken down comprehensively. Due to complex image upload logic discovered late in the sprint, US04 was partially completed and remaining sub-tasks were carried over to Sprint 2.

Individual Contributions

Member	Role	Key Contributions	Commits
P.M.A.D.N. Kulathunga	Product Owner	Backlog refinement, acceptance criteria, sprint planning, requirement documentation, Jira oversight	14
B.V.S.M. Athurupana	Scrum Master	Sprint ceremonies facilitation, impediment tracking, pipeline standardization, repository integration	12
R.R.M.G.H. Rathnayake	Frontend Dev	Login page, dashboard layout, vehicle list/form UI, React hooks, interface state validations	22
H.M.S.N. Rajapaksha	Backend Dev	Auth API, vehicle CRUD API routers, Prisma schema handling, middleware implementations	21
G.G.P.S. Gunathilaka	Full-Stack/QA	Auth integration testing, component test suites, code PR review, database container testing, UI fixes	20

Table 5: Sprint 1 Individual Contributions

Sprint Review

Completed Features

- **User Authentication:** Secure login and registration with JWT tokens and role-based access control for four user roles (Admin, Manager, Sales Agent, Inventory Staff)
- **Vehicle Inventory:** Full CRUD operations for vehicle listings with search, filter, and pagination capabilities seamlessly mapping API definitions.
- **Dashboard Layout:** Responsive sidebar navigation bridging optimal user visibility and summary metrics securely via Role-Based Access Controls.
- **Database:** Complete schema with seven entities formulated alongside Prisma deployment structures resolving all foundational setups perfectly.
- **API Structure:** Standardised architecture resolving strict response standards coupled beautifully with robust Zod object checking paradigms.

Incomplete / Carried Forward

- **US04: Edit and update vehicle details:** The backend endpoints successfully support modification, but frontend edge cases with removing and updating vehicle images triggered state latency issues. This required pausing the UI integration for these edge cases. The remaining tasks are carried forward to Sprint 2.

Demo Feedback

Software presentation resulted in a complete confirmation of structural layout design specifications matching MoSCoW parameters. Evaluators validated the seamless integration of Postgres/Docker deployments alongside robust React-driven login portals gracefully avoiding monolithic blockages.

Sprint Retrospective

What Went Well

- Separation of the Frontend client and Backend systems early enabled swift, parallel component programming minimizing deadlocked hours.
- Using Prisma significantly streamlined database queries circumventing boilerplate coding redundancy entirely keeping efficiency very high.
- Jira synchronization tasks were accomplished autonomously with programmatic integration reflecting perfectly equal workload and point distribution amongst group resources matching constraints.

What Could Be Improved

- Establishing and detailing REST API interaction logic mapping even quicker could offset interface dependency hang-ups early during testing phase iterations.
- Time allocations explicitly mapped for final deployment-level Quality Assurance procedures required broader attention avoiding crunch-level merge operations at milestones.

Action Items for Sprint 2

#	Action Item	Owner
1	Generate unified Swagger/Postman contracts defining all Sprint 2 API dependencies immediately	Backend Dev
2	Establish continuous PR peer review cycles limiting bottleneck accumulations at deadline thresholds	Scrum Master
3	Lock exactly 2 days exclusively for formalized test validations heading towards Sprint 2 conclusion	QA Dev

Table 6: Retrospective Action Items

Sprint 2 Preview

Sprint 2 Goal

Implement customer management, sales transaction workflow, appointment scheduling, and search/filter enhancements.

Planned User Stories

ID	User Story	Priority	Points
US05	As a sales agent, I want to create and manage customer profiles so that I can track interactions and preferences	Must	5
US06	As a sales agent, I want to record a vehicle sale with customer and payment details so that transactions are documented	Must	8
US07	As a customer, I want to book a test drive appointment so that I can evaluate a vehicle before purchasing	Must	5
US08	As a sales agent, I want to view my appointments and upcoming test drives so that I can prepare accordingly	Must	3

Table 7: Sprint 2 Planned Backlog